

Week 3

1. The UNIX File I/O System

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, September 15th, 2025



University of Windsor

COMP-2560 System Programming

Unix file Input/Output (System I/O)

School of Computer Science
University of Windsor

Instructor: Dr. Andreas Maniatis

Content

- 1 Unix I/O system calls
- 2 `open()` / `close()` system call
- 3 `read()` and `write()` system calls
- 4 `lseek()` system call

Unix I/O system calls

Unix I/O system calls

Most Unix I/O can be performed using the system calls:

open: to open a file

create?: to create a new file or rewrite an existing one

read: to read a number of bytes

write: to write a number of bytes

lseek: to explicitly position the file offset

close: to close a file

In contrast to the standard I/O functions, the Unix I/O system calls are **unbuffered** (what does it mean?).

File descriptors

The kernel refers to any open file by a **file descriptor**, a **nonnegative integer**.

File descriptors range from 0 through `OPEN_MAX`, the total number of files a process can open.

The standard input (`stdin`), standard output (`stdout`) and standard error (`stderr`) have descriptors `0`, `1` and `2` respectively. The symbolic constants, defined in `<unistd.h>`, for these values are `STDIN_FILENO`, `STDOUT_FILENO` and `STDERR_FILENO`.

Unix I/O system calls
open() system call
read() and write() system calls
lseek() system call

File descriptors

File descriptor	Purpose	POSIX name	<i>stdio</i> stream
0	standard input	STDIN_FILENO	<i>stdin</i>
1	standard output	STDOUT_FILENO	<i>stdout</i>
2	standard error	STDERR_FILENO	<i>stderr</i>

open() system call

open() synopsis

```
int open(const char *fName, int oflag[, int mode])
```

Returns file descriptor if OK, -1 otherwise.

The argument `oflag` is formed by bitwise **OR**'ing together 1 or more of the following constants (in `<fcntl.h>`)

- `RONLY`: Open for reading only
- `WONLY`: Open for writing only
- `RDWR`: Open for reading and writing only
- `APPEND`: Open for writing after the end of file
- `CREAT`: Create a file
- `_EXCL`: do your research in lab 1
-

Note that the third argument, only used when a file is created, supplies the initial file's permission flag settings, as an octal value.

Examples

Flag	Purpose
O_RDONLY	Open for reading only
O_WRONLY	Open for writing only
O_RDWR	Open for reading and writing
O_CLOEXEC	Set the close-on-exec flag (since Linux 2.6.23)
O_CREAT	Create file if it doesn't already exist
O_DIRECT	File I/O bypasses buffer cache
O_DIRECTORY	Fail if <i>pathname</i> is not a directory
O_EXCL	With O_CREAT: create file exclusively
O_LARGEFILE	Used on 32-bit systems to open large files
O_NOATIME	Don't update file last access time on <i>read()</i> (since Linux 2.6.8)
O_NOCTTY	Don't let <i>pathname</i> become the controlling terminal
O_NOFOLLOW	Don't dereference symbolic links
O_TRUNC	Truncate existing file to zero length
O_APPEND	Writes are always appended to end of file
O_ASYNC	Generate a signal when I/O is possible
O_DSYNC	Provide synchronized I/O data integrity (since Linux 2.6.33)
O_NONBLOCK	Open in nonblocking mode
O_SYNC	

Examples

```
if ((d=open("data.txt", O_RDONLY)) == -1)
    fatalError();
```

d=open(name, O_CREAT|O_WRONLY, 0640) In this case,
0700 means give all rights to the owner of this file and no
permission to any other user.

Example values for mode :

0400: Allow read by owner

0200: Allow write by owner

0100: Allow execute

0040: Allow read by group

0004: Allow read by others

0777: Allow read/write/execute by all

XXX XXX XXX:

we discussed this in class, i.e., file permission for self, group, others

Examples

```
/* Open existing file for reading */

fd = open("startup", O_RDONLY);
if (fd == -1)
    errExit("open");

/* Open new or existing file for reading and writing, truncating to zero
   bytes; file permissions read+write for owner, nothing for all others */

fd = open("myfile", O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR);
if (fd == -1)
    errExit("open");

/* Open new or existing file for writing; writes should always
   append to end of file */

fd = open("w.log", O_WRONLY | O_CREAT | O_TRUNC | O_APPEND,
          S_IRUSR | S_IWUSR);
if (fd == -1)
    errExit("open");
```

Examples

if `open()` succeeds, it is guaranteed to use the lowest-numbered unused file descriptor for the process.

We can use this feature to ensure that a file is opened using a particular file descriptor.

For example, the following sequence ensures that a file is opened using standard input (file descriptor 0).

```
if (close(STDIN_FILENO) == -1)      /* Close file descriptor 0 */  
    errExit("close");  
  
fd = open(pathname, O_RDONLY);  
if (fd == -1)  
    errExit("open");
```

read() and write() system calls

read() synopsis

```
ssize_t read(int fd, void *buf, size_t nbyte);
```

Reads as many bytes as it can, up to `nbyte`, and returns the number of bytes actually read.

`ssize_t` and `size_t` are usually defined as long integer (larger than an integer).

The value returned by `read()` can be:

-1: in case of an error

smaller than `nbyte`: the number of bytes left before the end of file was less than `nbyte` or, when reading from a keyboard where up to a line is normally read or, when reading from a network.

0: the end of file has been already reached

write() synopsis

```
ssize_t write(int fd, const void *buf, size_t nbyte);
```

`write` returns `nbyte` if OK and -1 otherwise.

Example of a copy utility (copy.c)

```
#include <unistd.h> #include <fcntl.h>

int main(int argc, char *argv[]){
    int fd1, fd2;
    char buffer[100];
    long int n1;

    if(((fd1 = open(argv[1], O_RDONLY)) == -1) || ((fd2
        = open(argv[2], O_CREAT|O_WRONLY|O_TRUNC,
            0700)) == -1)){
        perror("file problem ");
        exit(1);
    }

    while((n1=read(fd1, buffer, 100)) > 0)
        if(write(fd2, buffer, n1) != n1){
            perror("writing problem ");
            exit(3);
        }

    // Case of an error exit from the loop

    if(n1 == -1){
        perror("Reading problem ");
        exit(2);
    }
    Close(fd1);
    close(fd2);
    exit(0);
}
```

Homework:
Can you find the error?

Unix I/O system calls
open() system call
read() and write() system calls
lseek() system call

```
int close(int fd)
```

`int close(int fd)` frees the file descriptor `fd`.

If no other file descriptor is associated with a particular open file, the kernel frees all resources that were used for that file.

`close()` returns 0 when OK and -1 otherwise.

For example, -1 will be returned if `fd` was already closed.

lseek() system call

lseek() synopsis

```
off_t lseek(int fd, off_t offset, int whence);
```

Return the resulting offset if OK, -1 otherwise.

The return type `off_t` is a long integer.

Similar to `fseek()`, it sets the file pointer (position) associated with the open file descriptor specified by the file descriptor `fd` as follows:

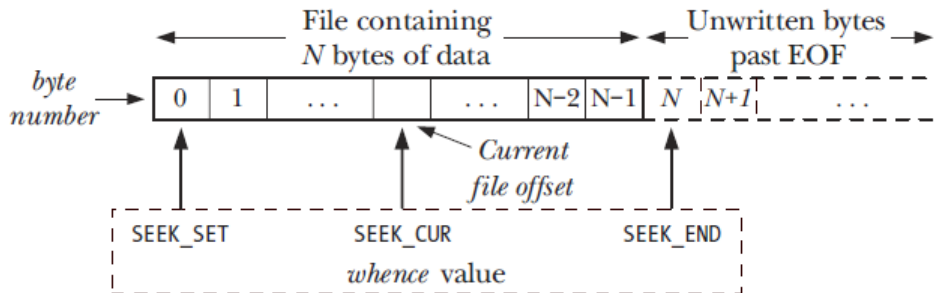
If `whence` is `SEEK_SET`, the pointer is set to offset bytes.

If `whence` is `SEEK_CUR`, the pointer is set to its current location plus offset.

If `whence` is `SEEK_END`, the pointer is set to the size of the file plus offset.

These three constant are defined in `<unistd.h>`.

...lseek()



```
lseek(fd, 0, SEEK_SET);      /* Start of file */
lseek(fd, 0, SEEK_END);      /* Next byte after the end of the file */
lseek(fd, -1, SEEK_END);     /* Last byte of file */
lseek(fd, -10, SEEK_CUR);    /* Ten bytes prior to current location */
lseek(fd, 10000, SEEK_END);  /* 10001 bytes past last byte of file */
```



```
Unix I/O system calls
open() system call
read() and write() system calls
lseek() system call
```

...lseek()

Note

`lseek()` allows the file pointer to be set beyond the existing data in the file. If data are later written at this point, subsequent reads in the gap between the previous end of data and the newly written data will return bytes of value 0 until data are written into the gap.

Example

Reversing a file

Write a C program, called reverse, to take a file as input then copies it to another file in reverse order. That is, the last byte becomes the first, the byte just before the last one becomes the second, etc.

The program call should look like:

```
reverse fileIn fileOut
```

There are two solutions,

either we use `lseek()` on `fileIn` (read bytes from the end), or

we use `lseek()` on `fileOut` (write bytes starting at the end)

```
// reversel.c
// solution 1: Start reading at the end
//          Use lseek() on fileIn

#include <unistd.h> #include <fcntl.h>

int main(int argc, char *argv[]){ int
fd1, fd2;
char buffer; // 1 character buffer
long int i=0, fileSize=0;

fd1=open(argv[1], O_RDONLY);
fd2=open(argv[2], O_CREAT|O_WRONLY|O_TRUNC, 0755);

while(read(fd1, &buffer, 1)>0)
    fileSize++;
while(++i <= fileSize){
    lseek(fd1, -i, SEEK_END);
    read(fd1, &buffer, 1);
    write(fd2, &buffer, 1);
}
close(fd1);
close(fd2);
}
```

Homework:
Can you find the error?

```
// reverse2.c
//solution 2: start writing at end
int main(int argc, char *argv[]){
    int fd1, fd2;
    char buffer; // 1 character buffer
    long int i=0, fileSize=0;

    fd1=open(argv[1], O_RDONLY);
    fd2=open(argv[2], O_CREAT|O_WRONLY|O_TRUNC, 0755);

    while(read(fd1, &buffer, 1)>0)
        fileSize++;
    lseek(fd2, fileSize-1, SEEK_SET); //pointer to end
    lseek(fd1, 0, SEEK_SET); // rewind fd1!!
    while(++i <= fileSize){
        read(fd1, &buffer, 1);
        lseek(fd2, -i, SEEK_END);
        write(fd2, &buffer, 1);
    }
    close(fd1);
    close(fd2);
}
```

```
Unix I/O system calls  
open() system call  
read() and write() system calls  
lseek() system call
```

`lseek()` again.

The file's offset can be greater than the file's current size, in which case the next write to the file will extend the file. This is referred to as creating a hole in a file and is allowed.

Any bytes in a file that have not been written are read back as 0.

See the example code "`hole.c`"

Run it and inspect its contents using `od -c file.hole`

errno and perror (char *) library functions

Unix/Linux provides a globally accessible integer variable which contains an error code number ...

Remember to include `errno.h` before using it.

`perror(char *)` is a library routine to output error message related to `errno` ...

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OQUTJPL Q DZC
NYDWIGLIPJL QUTJX

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRMVQJXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYULM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111001100
00000111011011
11001100110011

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor